Flight Software Architecture

1. Design Philosophy and Assumptions
   1.1 FreeRTOS
   1.2 Propulsion
   1.3 Communication and Driver Independence
   1.4 Boot and OTA Firmware Update as Core Architecture
2. Architectural Approach
   2.1 Layered Architecture
   2.2 Core Framework — cFE
   2.3 FreeRTOS Adaptation of cFE
   2.4 Application Organisation
   2.5 Application and Data Flow
   2.6 Software Bus and External Networks
3. Software Components
   3.1 Executive Services
   3.2 Software Bus
   3.3 Event Services
   3.4 Time Services
   3.5 Table Services
   3.6 Mode Manager
   3.7 Command Ingest
   3.8 Telemetry Output
   3.9 Mission Scheduler
   3.10 AOCS Application
   3.11 Bootloader and OTA Manager
   3.12 Driver-Independent Subsystem Library and Software Bus Integration
4. Scheduling and Real-Time Behaviour
5. Communication Architecture
   5.1 Subsystem communication
   5.2 UHF and S-band

6. Fault Containment and Recovery

1. Design Philosophy and Assumptions

Keeping in mind the expected functions of this OBC, especially real-time AOCS control and propulsion supervision, the system needs a deterministic execution model. For that reason, the baseline proposed here is a hard real-time, single-core architecture running FreeRTOS.

The single-core decision is mainly driven by the size of the team, the fixed launch schedule and the advantages of having an architecture that is easier to understand, profile and verify. The complete critical workload should be schedulable on one core without relying on parallel execution.

FreeRTOS is selected because it is lightweight, open source and familiar to many software developers. Its default single-core scheduler is fixed-priority and preemptive: the highest-priority ready task executes, while lower-priority work can run when critical tasks are blocked waiting for their next release or for an event.

Other operating systems remain technically possible:

- **RTEMS** has strong space heritage and is a valid alternative, but it generally requires a larger integration and configuration effort than the small FreeRTOS baseline proposed here.
- **VxWorks** is also suitable for critical embedded systems and can provide commercial support and qualification material, but licensing cost and team experience would need to justify its use.
- **ChibiOS** is a compact RTOS that could support a smaller embedded platform, although FreeRTOS is preferred because it is more familiar to the proposed team.
- **Linux with PREEMPT_RT** could be considered if the platform later required complex networking, large filesystems or payload-processing frameworks. It is not selected for the primary control processor because the AOCS and propulsion paths benefit from a smaller and more predictable runtime environment.

Regardless of the selected RTOS, the mission applications should not call the native operating-system API directly. The platform will be built over a small **Operating System Abstraction Layer**, or OSAL, which provides only the services required by the flight software:

- Tasks
- Queues
- Mutexes
- Binary and counting semaphores
- Task notifications / Software ISR
- Software timers
- Monotonic time

This does not make a future RTOS replacement automatic. A change of RTOS would still require timing analysis, driver adaptation and revalidation. However, it prevents operating-system-specific calls from spreading through the mission applications and keeps the changes concentrated in the OSAL, Platform Support Package and low-level drivers.

1.1 FreeRTOS

FreeRTOS is selected because the system includes hard real-time functions, particularly AOCS execution and propulsion sequencing.

If the propulsion subsystem is going to be a standalone product with its own controller, it will internally handle low-level actuation and accept high-level commands, time-tagged operations or PPS synchronisation from the OBC.

This reduces the low-level propulsion timing handled by the OBC, but the OBC must still provide deterministic command sequencing, supervision and recovery coordination.

The FreeRTOS scheduler will use a fixed-priority, pre-emptive configuration. This makes the execution model relatively simple and analysable:

- The highest-priority ready task executes.
- AOCS and propulsion-related tasks can pre-empt telemetry, storage and background processing.
- Each periodic task can have a defined release rate, deadline and maximum permitted execution time.
- Worst-case execution time can be measured on the target hardware.
- Stack use, queue use, task execution and deadline misses can be monitored.
- The behaviour can be analysed without depending on a complex general-purpose process scheduler.

The system is assumed to use one processor core with enough throughput for the full nominal workload and the required recovery margin.

A multicore CPU can provide more processing capacity, but it also introduces:

- Race conditions
- Synchronisation complexity
- More complex worst-case execution-time analysis
- Possible interference between critical and non-critical workloads

Multicore operation does not make timing analysis impossible, but it makes the analysis and verification significantly more difficult. If a multicore processor is eventually selected, the safety-critical control path should remain independently schedulable on one core. Additional cores could then be used for payload processing, data compression or other non-critical functions.

Timing measurements should be performed on the target processor. CI/CD can then be used to detect timing regressions by comparing execution-time measurements, stack high-water marks and hardware-in-the-loop results against established limits. The build server itself cannot prove the target WCET, but it can automatically reject changes that exceed previously defined timing or resource thresholds during target-based testing.

FreeRTOS+CLI can also be integrated as an optional diagnostic interface. It is a small command-interpreter. It can be connected to UART, USB, Ethernet or another test interface and used during development and AIT.

The CLI should not become a second uncontrolled command path. Safety-critical commands must pass through the same validation, mode checks and authorisation rules as commands received through the nominal TM/TC interface. The CLI is disabled in the flight configuration.

1.2 Propulsion

Two propulsion-control approaches have been considered.

The first approach is direct actuation from the OBC. In this architecture, the OBC controls valve timing, actuator sequencing and immediate shutdown.

The second approach uses a dedicated Propulsion Control Unit, or PCU/ECU, while the OBC sends high-level commands and supervises the operation.

The selected baseline is the second approach: the propulsion system provides its own dedicated control unit, and the OBC interfaces with it through a defined command-and-telemetry protocol.

This is the most suitable boundary when the propulsion unit is intended to operate as a standalone subsystem or product. It also provides better fault containment and separates mission-level autonomy from low-level actuator timing.

The propulsion control unit is responsible for:

- Low Level actuator (valves, heaters, sensors), and timing.
- Local watchdog supervision
- Fast actuator-level fault detection, immediate shutdown when required.
- Returning the propulsion system to a safe state if communication with the OBC is lost

The OBC is responsible for:

- Arming and disarming
- Mission-level sequencing
- Spacecraft-mode checks
- Inhibit management
- Command

- Telemetry assessment
- Fault reporting
- Recovery coordination
- Coordination with AOCS, thermal and power constraints

The interface between the OBC and the propulsion controller will use a command-and-telemetry protocol. Additional discrete signals may be used where appropriate, for example:

- PPS or synchronisation input
- Hardware inhibit
- Ready indication
- Safe-state indication
- Fault indication
- Emergency shutdown

The OBC commands propulsion operations according to the AOCS constraints, propulsion requirements, spacecraft modes and FDIR specification. Operations may originate from ground commands, the Mission Scheduler or authorised autonomous logic.

If direct control of the propulsion valves is required, the architecture can support it. However, the task controlling the valves would become one of the highest-criticality tasks in the OBC and would require:

- Dedicated hardware timers
- Strict worst-case execution-time analysis
- Independent watchdogs
- Direct actuator feedback (more cables, which are bad for low latency)
- Bounded interrupt latency
- Additional software and hardware safety barriers
- More extensive verification

For this reason, direct valve control is not selected as the baseline for this example.

1.3 Communication and Driver Independence

Mission applications and reusable libraries should not depend directly on low-level communication protocols, hardware abstraction layers or operating-system calls.

For example, the Propulsion Application should not open a UART or call a CSP transaction directly. It should publish a normalised command through the Software Bus // use a stable subsystem API

A representative mission-facing call is:

```
        propulsion_status_t propulsion_command_send(
            const propulsion_command_t *command);
```

The implementation below this API may use any communication stack that is implemented for it.

The separation is:

```
Mission Application
        ↓
cFE Software Bus message
        ↓
Subsystem Interface Application
        ↓
Subsystem-specific API
        ↓
Protocol encoder / decoder
        ↓
Generic transport interface
        ↓
Platform-specific driver
```

This is important because the Software Bus and the driver abstraction solve different problems:

- The **Software Bus** decouples independently scheduled flight applications.
- The **subsystem library** hides the device command and telemetry protocol.
- The **transport interface** hides whether the protocol uses UART, CAN, CSP, SpaceWire or another transport.
- The **platform driver** hides the operating system and peripheral implementation.

The same mission application can therefore run in a host simulation, on development hardware and on the final flight processor. It also becomes possible to replace a subsystem bus without rewriting the complete mission architecture.

The complete Software Bus and driver path is described in Sections 2.5 and 3.13.

1.4 Boot and OTA Firmware Update as Core Architecture

Firmware Update over the Air, or OTA FWU, is treated as a core part of the flight architecture. This is particularly important when software development continues close to a fixed launch date.

The boot and update architecture must tolerate:

- Interrupted uploads

- Corrupted files
- Invalid configurations
- Incompatible software
- Failed startup
- Repeated watchdog resets
- An image that boots but does not operate correctly

The baseline therefore includes:

- A protected bootloader
- Signed software images
- Image-integrity validation
- Persistent reset and boot metadata
- A Golden Recovery Image
- A/B operational images when storage permits
- Automatic rollback
- Cooperation with the hardware watchdog and power system

The bootloader should be small and changed only when strictly necessary.

Its main responsibilities are:

1. Read the reset reason and persistent boot metadata.
2. Select the image to execute.
3. Validate the selected image header, integrity and signature.
4. Start the selected image.
5. Fall back to another image after repeated boot failures, either B or Golden.

The communication, file-transfer and repair functionality belongs in a separate Golden Recovery Image. The recovery image contains the minimum software required to:

- Communicate with ground through at least the contingency link
- Access the interfaces required for spacecraft safing and recovery
- Read reset and boot information
- Access non-volatile storage
- Receive a replacement software image
- Validate an uploaded image
- Select the image for the next boot
- Restore a known configuration

- Reboot the OBC

The recovery image does not need to execute nominal payload operations or the complete AOCS implementation. It may contain only the minimum control required to keep the spacecraft safe and recoverable.

If sufficient non-volatile memory is available, the preferred layout is:

```
Protected Bootloader
Golden Recovery Image
Operational Image A
Operational Image B
Persistent Boot Metadata
Configuration and Mission Storage
```

In this configuration, one operational slot remains validated while the other receives the candidate image. The previous operational image is preserved until the new image confirms that it is healthy.

In the reduced layout, the Golden image provides the recovery path. The system does not retain two complete operational releases.

Software transfer should be block-based and resumable.

A complete CFDP or file-transfer implementation may be used (from CFE) , but a smaller mission-specific protocol is also acceptable if it supports:

- Per-block integrity checking
- Resume after loss of contact
- Final complete-image verification

After reception, the complete image is checked using:

- Cryptographic hash
- Entry point and memory limits

A commit identifier can be stored as build metadata, and a fast checksum such as xxHash may be useful during development or block handling.

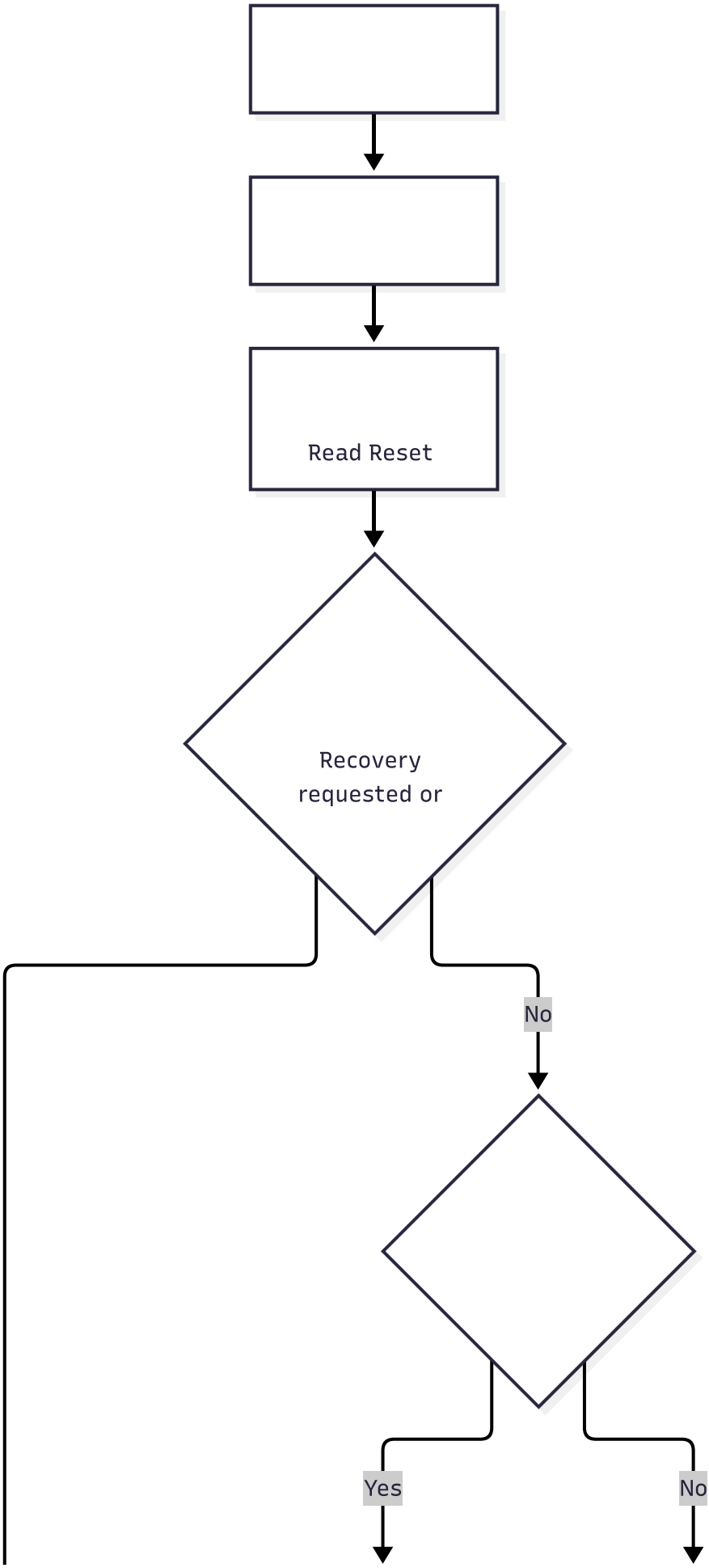
The image is first marked as a candidate. After reboot, it must complete a health-confirmation sequence before it is committed as the active image. Reaching `main()` or starting the FreeRTOS scheduler is not enough.

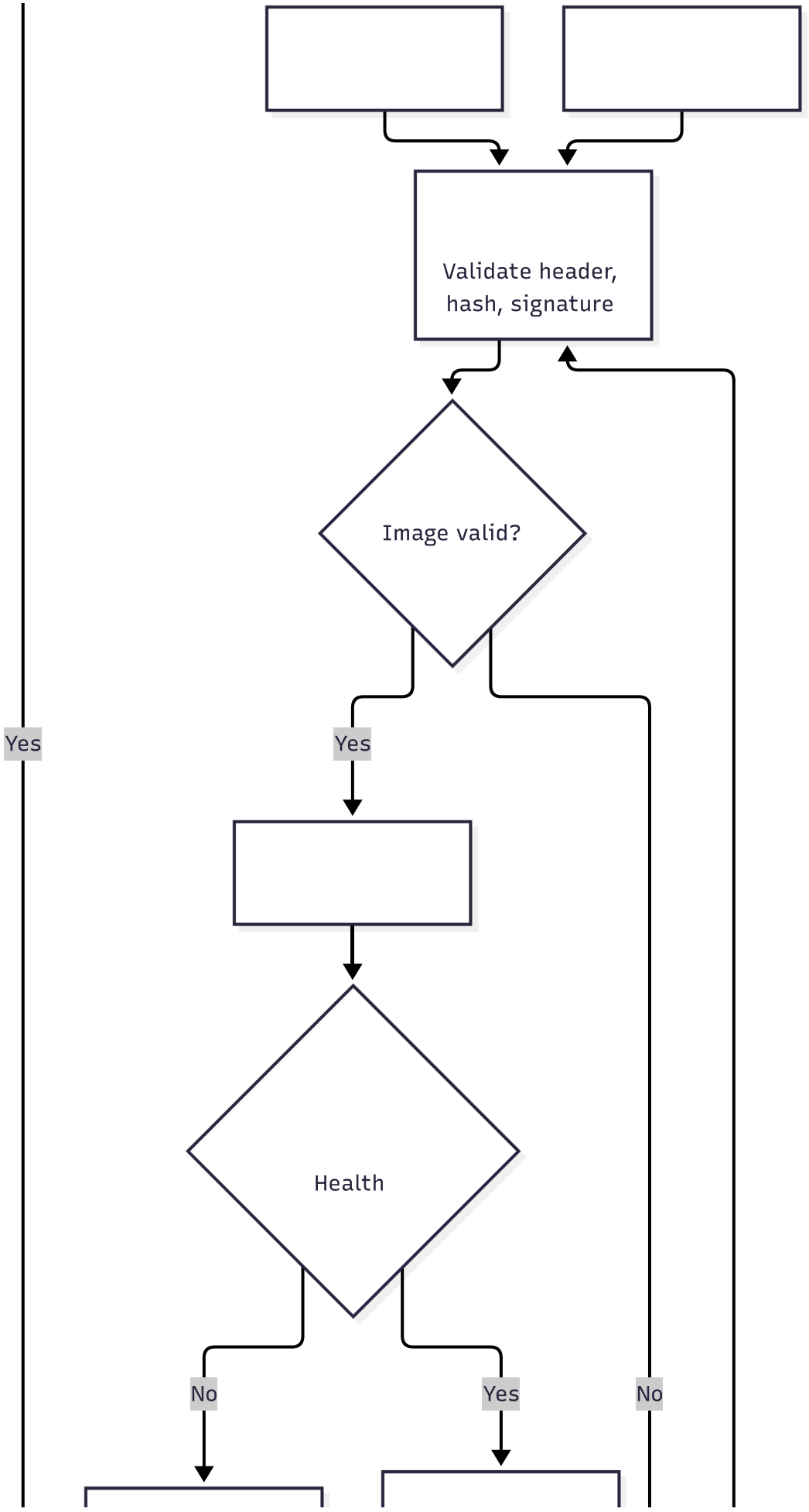
Health confirmation should require, at minimum:

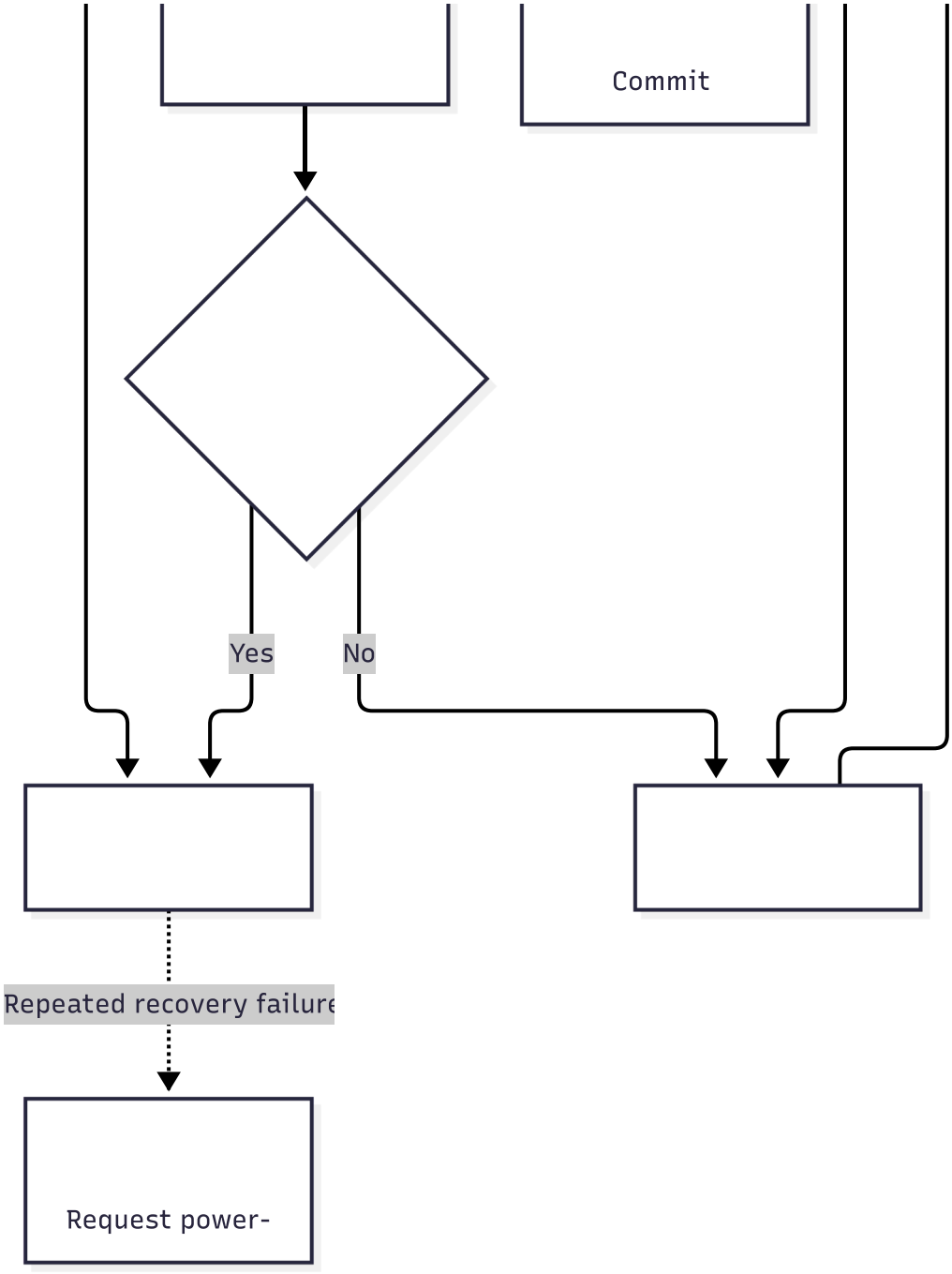
- Successful cFE core initialisation
- Running Mode Manager
- Running Health Monitor

- Stable execution for a defined period
-

If the candidate does not confirm itself before a defined timeout, or if it causes repeated watchdog resets, the bootloader selects the previous validated image or the Golden Recovery Image.


Read Reset
Recovery
requested or
No
Yes
No


Validate header, hash, signature
Image valid?
Yes
Yes
Health
No
Yes
No



This protected-bootloader, candidate-image and confirmation pattern is also used in common embedded update systems such as MCUboot and FreeRTOS OTA implementations. The details must still be adapted to the selected processor, flash technology and spacecraft operations concept.

> [!assumption] Platform
> The OBC uses a flight-capable single-core embedded processor with hardware timers, an independent watchdog, non-volatile memory and standard spacecraft interfaces.

> [!assumption] Application loading
> The first FreeRTOS flight configuration uses statically linked applications or a controlled static application registry. Runtime ELF loading is not required for the initial baseline.

> [!assumption] Memory protection
> ECC-protected RAM and non-volatile storage are preferred where supported by the processor and memory technology. Software checks can provide additional corruption detection, but they are not equivalent to hardware ECC and consume CPU and memory resources.

> [!assumption] Communications
> A robust low-rate radio link is available for nominal operations and contingency access. A higher-rate link may be available for payload data, detailed logs and software transfer. The high-rate link may require more accurate pointing, while the low-rate link should remain available in degraded or safe modes.

> [!assumption] Propulsion
> The propulsion system provides a command-and-telemetry interface through a dedicated controller or interface electronics. The OBC performs mission-level FDIR and recovery coordination, while immediate actuator-level failures and shutdown actions are handled and logged locally by the propulsion controller.

> [!assumption] Recovery
> The power subsystem can detect repeated resets or missed watchdog servicing and can support escalation to a recovery boot mode. This may be performed autonomously or through a ground-commanded OBC power cycle.

---

2. Architectural Approach

The architecture I am most comfortable with is divided into six logical layers:

1. **Mission applications**
2. **Common mission services and reusable libraries**
3. **cFE-based core**
4. **Subsystem, COTS-device and communication libraries**
5. **Operating-system and hardware abstraction layers**
6. **FreeRTOS, board-support software and hardware**

The intended dependency direction is mainly downward:

```
Mission Applications
        ↓
Common Mission Services
        ↓
cFE-Based Core
        ↓
Subsystem and Communication Libraries
        ↓
OSAL / PSP / HAL
        ↓
FreeRTOS / BSP / Hardware
```

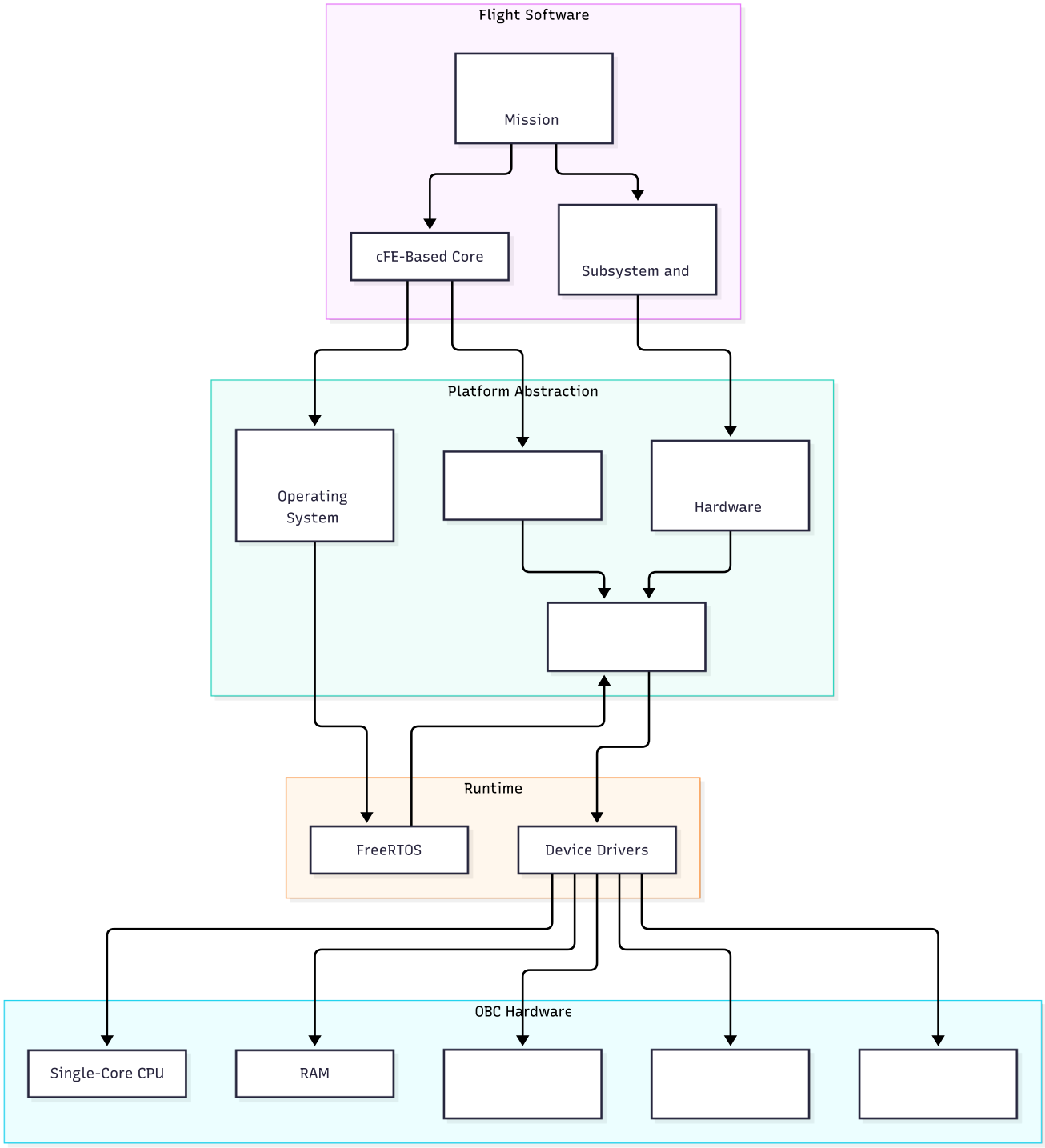
Mission applications should not directly access:

- Native FreeRTOS primitives
- Peripheral registers
- UART, CAN or any comm drivers
- PUS packet definitions
- Filesystem implementations
- COTS-specific packet formats

These dependencies are isolated behind stable APIs and Software Bus messages.

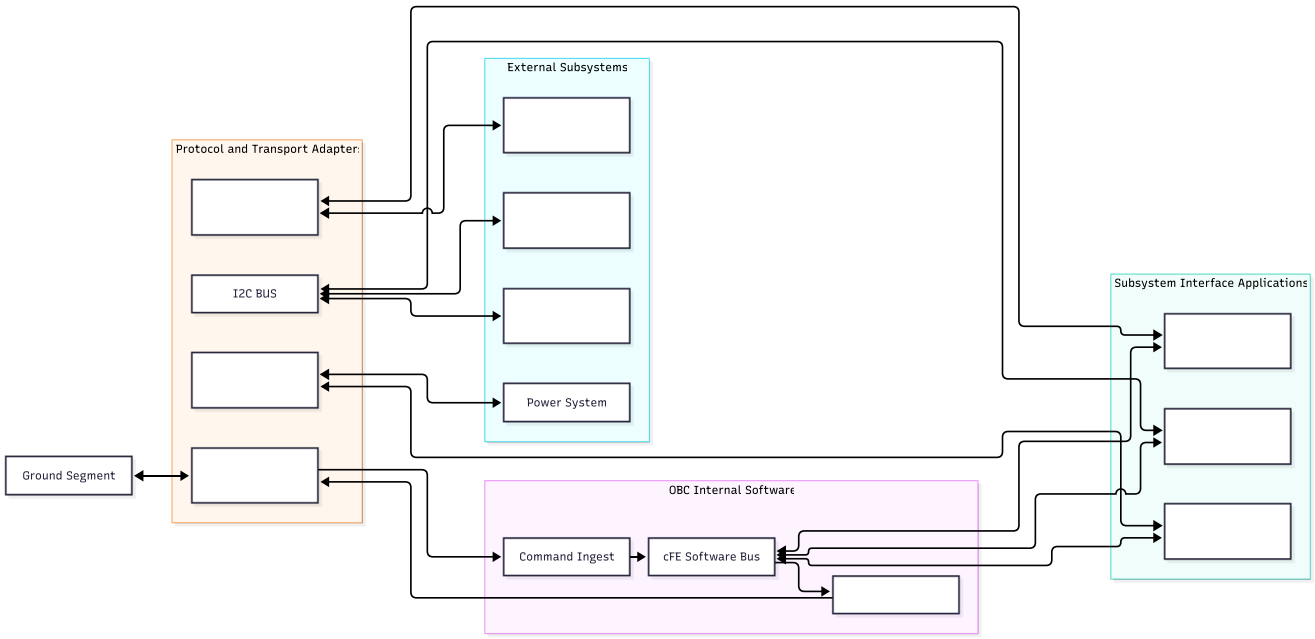
This allows mission logic to be developed and tested independently from the final hardware. It also makes it possible to replace a subsystem, communication bus or processor without redesigning the complete flight software.

2.1 Layered Architecture



The diagram is logical rather than hardware-specific. The Propulsion Interface may use CSP over CAN in one implementation and a direct UART or CAN protocol in another. The Propulsion Application remains unchanged because the transport and protocol implementation are hidden below the Software Bus interface.

From a CDH point of view :



2.2 Core Framework — cFE

The reusable software core could also be provided by a commercial spacecraft SDK, including platforms from suppliers such as GomSpace, Space Inventor or Bright Ascension. These solutions may reduce integration effort when the software is closely coupled to the supplier's OBC, but they may also increase dependency on a particular vendor, hardware platform or workflow.

Two relevant open-source flight-software frameworks are NASA's core Flight System, or cFS, and JPL's F Prime.

Both provide reusable flight-software infrastructure, although they use different architectural models:

- F´ follows a component-based C++ architecture with typed ports and generated deployment topologies.
- cFS uses a layered architecture based on cFE, OSAL, PSP and mission applications communicating through the Software Bus.

https://digitalcommons.usu.edu/cgi/viewcontent.cgi?article=6013&context=smallsat

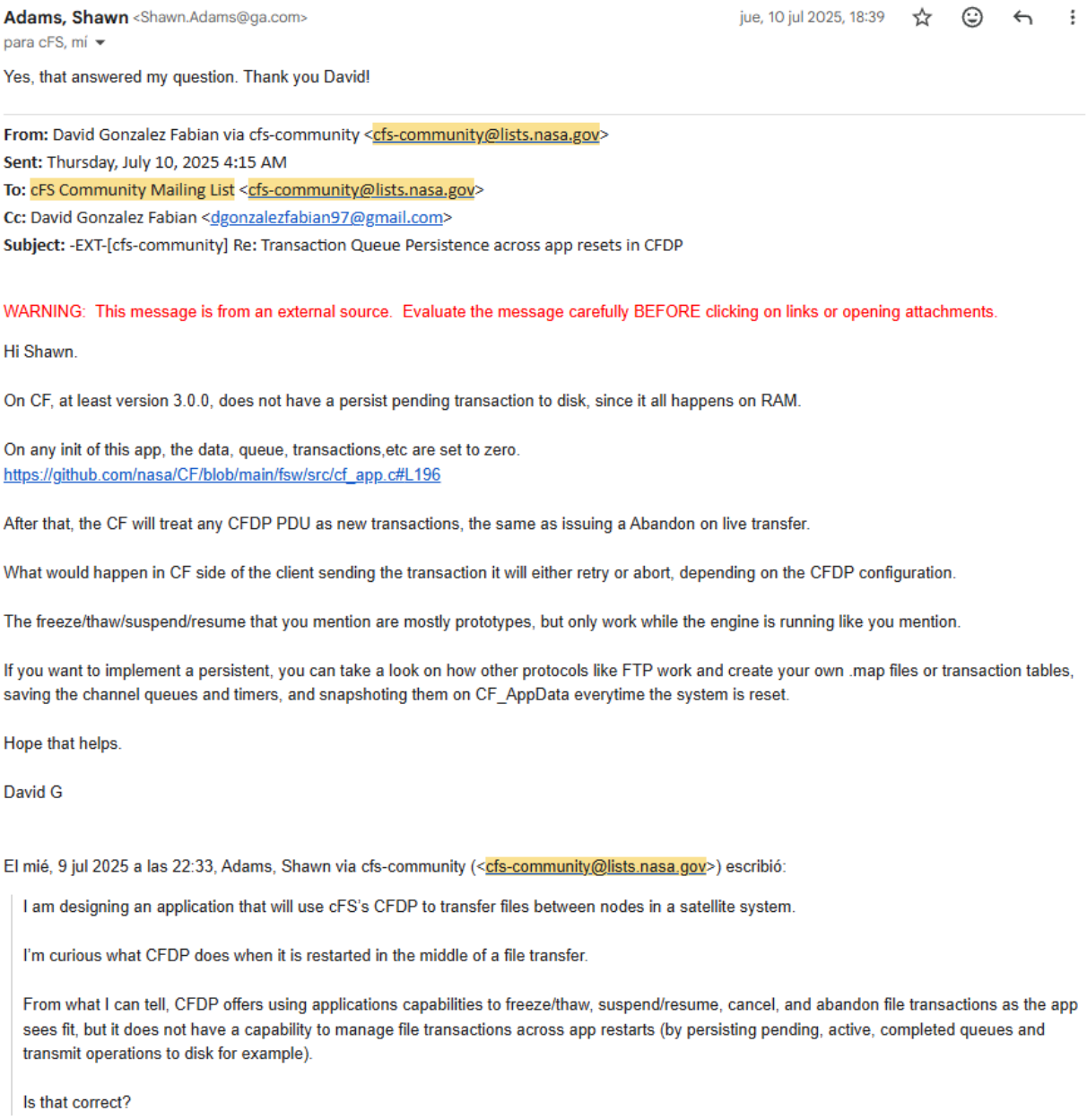
This paper compares the two of them.

For this architecture, I would select a **cFE-based core** rather than integrating the complete cFS bundle unchanged.

This choice is based on:

- Previous experience with parts of cFS and cFE.
- The suitability of the Software Bus for modular flight software
- Clear separation between applications, operating system and hardware

- The ability to run applications on host and target platforms, which eases development a lot
- The option to select only the services required by the mission
- Existing user and developer community, which can help and provide info when needed, really OSS friendly



Adams, Shawn <Shawn.Adams@ga.com> jue, 10 jul 2025, 18:39

para cFS, mí

Yes, that answered my question. Thank you David!

From: David Gonzalez Fabian via cfs-community <cfs-community@lists.nasa.gov>
Sent: Thursday, July 10, 2025 4:15 AM
To: cFS Community Mailing List <cfs-community@lists.nasa.gov>
Cc: David Gonzalez Fabian <dgonzalezfabian97@gmail.com>
Subject: -EXT-[cfs-community] Re: Transaction Queue Persistence across app resets in CFDP

WARNING: This message is from an external source. Evaluate the message carefully BEFORE clicking on links or opening attachments.

Hi Shawn.

On CF, at least version 3.0.0, does not have a persist pending transaction to disk, since it all happens on RAM.

On any init of this app, the data, queue, transactions,etc are set to zero.
https://github.com/nasa/CF/blob/main/fsw/src/cf_app.c#L196

After that, the CF will treat any CFDP PDU as new transactions, the same as issuing a Abandon on live transfer.

What would happen in CF side of the client sending the transaction it will either retry or abort, depending on the CFDP configuration.

The freeze/thaw/suspend/resume that you mention are mostly prototypes, but only work while the engine is running like you mention.

If you want to implement a persistent, you can take a look on how other protocols like FTP work and create your own .map files or transaction tables, saving the channel queues and timers, and snapshoting them on CF_AppData everytime the system is reset.

Hope that helps.

David G

El mié, 9 jul 2025 a las 22:33, Adams, Shawn via cfs-community (<cfs-community@lists.nasa.gov>) escribió:

> I am designing an application that will use cFS's CFDP to transfer files between nodes in a satellite system.
>
> I'm curious what CFDP does when it is restarted in the middle of a file transfer.
>
> From what I can tell, CFDP offers using applications capabilities to freeze/thaw, suspend/resume, cancel, and abandon file transactions as the app sees fit, but it does not have a capability to manage file transactions across app restarts (by persisting pending, active, completed queues and transmit operations to disk for example).
>
> Is that correct?

Training : https://github.com/cfs-tools/cfs-basecamp/wiki/

The public cFS bundle is a starting point and example integration, not a complete flight distribution. A systems/arquitect is still responsible for selecting, adapting and verifying the required components.

The selected cFE services, which I would include in any OBC are :

- **Executive Services**, or ES
- **Software Bus**, or SB

- **Event Services**, or EVS
- **Time Services**, or TIME
- **Table Services**, or TBL
- **File Services API**, or FS

Command Ingest and Telemetry Output are also implemented as applications connected to the Software Bus,.

This creates a small reusable core that can support future missions while avoiding unnecessary framework weight.

The cFE approach is not monolithic. Applications are independently scheduled, expose defined interfaces and communicate mainly through Software Bus messages. This is consistent with the cFE application model, which distinguishes reusable software-only applications from hardware-servicing applications and recommends using published interfaces rather than internal calls between applications.

2.3 FreeRTOS Adaptation of cFE

The cFS architecture separates operating-system-specific functionality through OSAL and platform-specific functionality through the PSP.

For this mission, the intention is not to port every cFS application.

The OSAL implementation provides the FreeRTOS mechanisms required by these services. The PSP provides the processor- and board-specific functions, including:

- Early hardware initialisation
- Reset-reason access
- Watchdog control
- Non-volatile boot metadata
- Processor exceptions
- Hardware timers
- Memory-map definitions
- Board-specific startup logic

The preferred approach is:

1. Start from the published cFE, OSAL and PSP interfaces.
2. Identify the exact services required by the mission.
3. Implement and verify the required FreeRTOS OSAL backend.
4. Implement the processor- and board-specific PSP.
5. Avoid modifying cFE core behaviour unless a requirement makes it necessary.

6. Measure CPU, RAM, stack, queue and storage use on the target hardware.
7. Use the cFS Test Framework,

https://www.mdpi.com/2079-9292/15/5/1020

This paper goes insight of the cFS port into FreeRTOS.

2.4 Application Organisation

To avoid confusion with the cFE core and with processor cores, the main groups of flight applications are described as **application suites**.

The proposed organisation contains three principal suites:

1. **Mission and Data-Handling Application Suite**
2. **Subsystem Interface Application Suite**
3. **AOCS and Propulsion Application Suite**

These suites are logical software groupings. They are not separate processor cores.

MISSION AND DATA-HANDLING APPLICATION SUITE (THE CDH-CORE)

This suite handles:

- Spacecraft mode management
- Mission scheduling
- Ground-command coordination
- Payload operations
- Payload-data buffering
- File storage
- Downlink queueing
- Software update
- General FDIR coordination

Its main non-critical data path is:

```
Payload data reception
→ RAM buffering
→ Non-volatile storage
→ File integrity checks
→ Downlink queueing
→ S-band or selected link
```

Payload processing, file access and downlink preparation execute at a lower priority than AOCS and propulsion-related tasks.

SUBSYSTEM INTERFACE APPLICATION SUITE (THE DRIVER-CORE)

This suite manages communication with hardware and COTS subsystems such as:

- Reaction wheels
- IMUs
- Star trackers
- Sun sensors
- Magnetometers
- Magnetorquers
- Propulsion controller
- Power system
- Radios
- Payload electronics

It handles:

- Software Bus subscriptions
- Command encoding
- Telemetry decoding
- Transaction timeouts
- Protocol sequence counters
- Link-level retries
- Data buffering
- Timestamping on reception
- Communication-health monitoring
- Conversion into common internal data types

The Subsystem Interface Applications publish normalised data onto the Software Bus.

For example, the AOCS Application receives a common angular-rate message rather than a vendor-specific IMU packet. If the IMU is replaced, only the interface application and device library need to change, provided they continue publishing the same internal message type.

AOCS AND PROPULSION APPLICATION SUITE (THE RUNNER-CORE)

This suite executes the critical control and propulsion-supervision paths.

The AOCS path is:

```
Sensor acquisition
→ Validation and timestamping
→ State estimation
→ Control
→ Limit and inhibit checks
→ Actuator command
→ Deadline and health monitoring
```

The propulsion-supervision path is:

```
Operation request
→ Mode and inhibit validation
→ Sequence preparation
→ Propulsion-controller command
→ Execution monitoring
→ Result verification
→ Completion or fault reporting
```

The AOCS Application may execute autogenerated C code produced by Embedded Coder or another model-based development environment.

The autogenerated algorithm code should remain separated from platform-specific software. A manually developed wrapper provides:

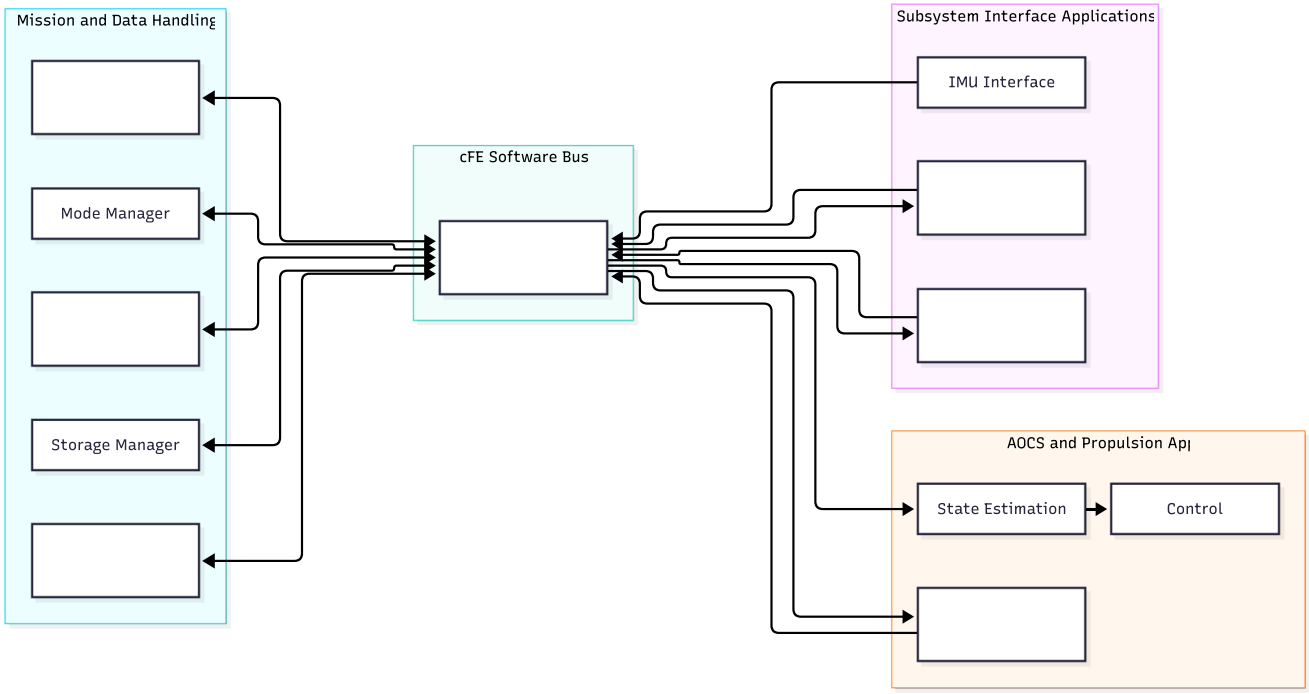
- Input conversion
- Output conversion
- Input validity checks
- Output saturation
- Timing measurements
- Model initialisation
- Software Bus interface
- FDIR interface

Protection mechanisms outside the control model remain in manually developed and independently testable software. These mechanisms may include:

- Input-age checks
- Invalid-sensor rejection
- Output saturation

- Actuator inhibits
- Fallback-state selection
- Hard-fault response

2.5 Application and Data Flow



An application does not need to know the implementation details of another application.

For example:

1. The AOCS Application publishes a reaction-wheel torque request.
2. The Reaction Wheel Interface subscribes to that message.
3. The interface converts the request into the wheel-specific command.
4. The command is transferred over the selected physical interface.
5. Reaction-wheel telemetry is decoded.
6. The interface publishes normalised telemetry on the Software Bus.
7. AOCS, Health Monitoring and Telemetry Output may subscribe to the same telemetry message.

2.6 Software Bus and External Networks

Direct application-to-application function calls should be limited to tightly coupled library functions executing within the same application context.

Operational communication between independently scheduled applications should pass through the Software Bus.

This provides:

- Observable message flow
- Easier testing
- Independent application restart
- Reduced coupling
- Protocol independence
- Clear data ownership
- Easier addition of monitoring applications
- Easier replacement of subsystem implementations

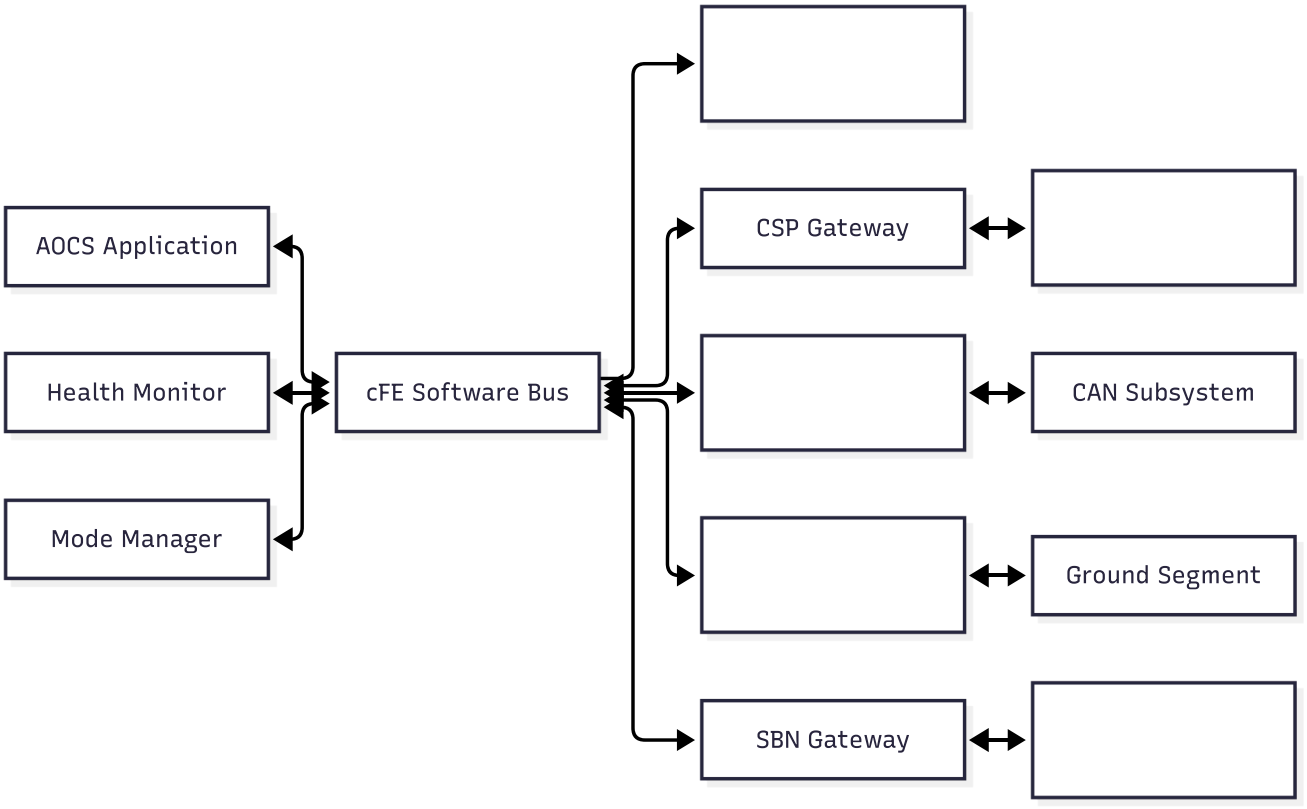
The Software Bus is an internal software messaging mechanism. It is not itself a physical CAN, SpaceWire or Ethernet bus.

External communication requires a gateway or subsystem interface application:

If another processor or spacecraft node also uses cFE, NASA’s Software Bus Network application is an example of bridging Software Bus traffic between separate cFS instances.

If the remote node does not use cFE, a mission-specific CSP, CANopen, SpaceWire or serial gateway can provide the equivalent function.

Only the messages required by the external node should be forwarded. It is normally not appropriate to transmit all internal Software Bus traffic onto a constrained physical bus.


AOCS Application
Health Monitor
Mode Manager
cFE Software Bus
CSP Gateway
CAN Subsystem
Ground Segment
SBN Gateway

3. Software Components

3.1 Executive Services

Executive Services manage the lifecycle of the cFE applications after the operational or recovery image has started.

The bootloader and Executive Services are related through reset state and startup information, but they are not the same component:

- The bootloader runs before FreeRTOS and cFE and selects the image to execute.
- Executive Services run inside the selected image and manage cFE application startup and lifecycle.
- The Update Manager receives and validates a future image while the operational software is running.

Executive Services are responsible for:

- Application startup order
- Application registration
- Initialisation status
- Application lifecycle management
- Controlled application restart
- Critical-application monitoring support
- Processor-reset requests
- Shutdown requests
- Critical Data Store access, where used
- System and application status reporting

Executive Services should not contain all Mode Manager or FDIR policy. Instead:

- ES provides mechanisms such as application status, restart and processor reset.
- Health Monitoring and FDIR decide when recovery is required.
- The Mode Manager authorises spacecraft-level state transitions.

A representative startup sequence is:

```
Bootloader selects image
→ FreeRTOS and BSP start
→ OSAL and PSP initialise
→ cFE core services start
→ Communication and storage services start
→ Mode Manager and Health Monitor start
→ Subsystem interfaces start
→ AOCS and propulsion applications start
→ Payload and background applications start
```

Each application should report whether initialisation succeeded. A failed return value from a critical startup stage should generate an event and a defined recovery action rather than leaving the system in a partially initialised state.

NASA’s cFS Health and Safety application is a useful reference for application monitoring, event monitoring, watchdog servicing and execution-counter reporting. The proposed mission may reuse it, tailor it or implement a smaller equivalent application.

3.2 Software Bus

The Software Bus is the central internal communication mechanism.

Applications subscribe only to the message identifiers they require. This avoids direct dependencies between independently scheduled applications.

Representative message groups include:

- Spacecraft mode and requested transitions
- Time status and synchronisation quality
- Sensor measurements
- Sensor validity and age
- AOCS state
- Actuator commands
- Propulsion commands and telemetry
- Power-system telemetry
- Health events
- Ground commands
- Command-verification reports
- Payload status
- Storage status

- Update status

Messages may also be subscribed to by:

- Telemetry Output
- Data Storage
- Health Monitoring
- Diagnostic or AIT tools
- External gateways

This makes it possible to record or forward selected internal data without changing the application that produces it. NASA’s Data Storage application is an existing example: it subscribes to Software Bus messages and stores selected messages in files for later transfer.

The Software Bus uses pipes with configured depths and per-message limits.

A pipe overflow or message overrun is a local error on the affected pipe. It does not automatically block all other subscribers. The Software Bus reports the condition, and the consuming application and Health Monitor can include the counters in FDIR decisions.

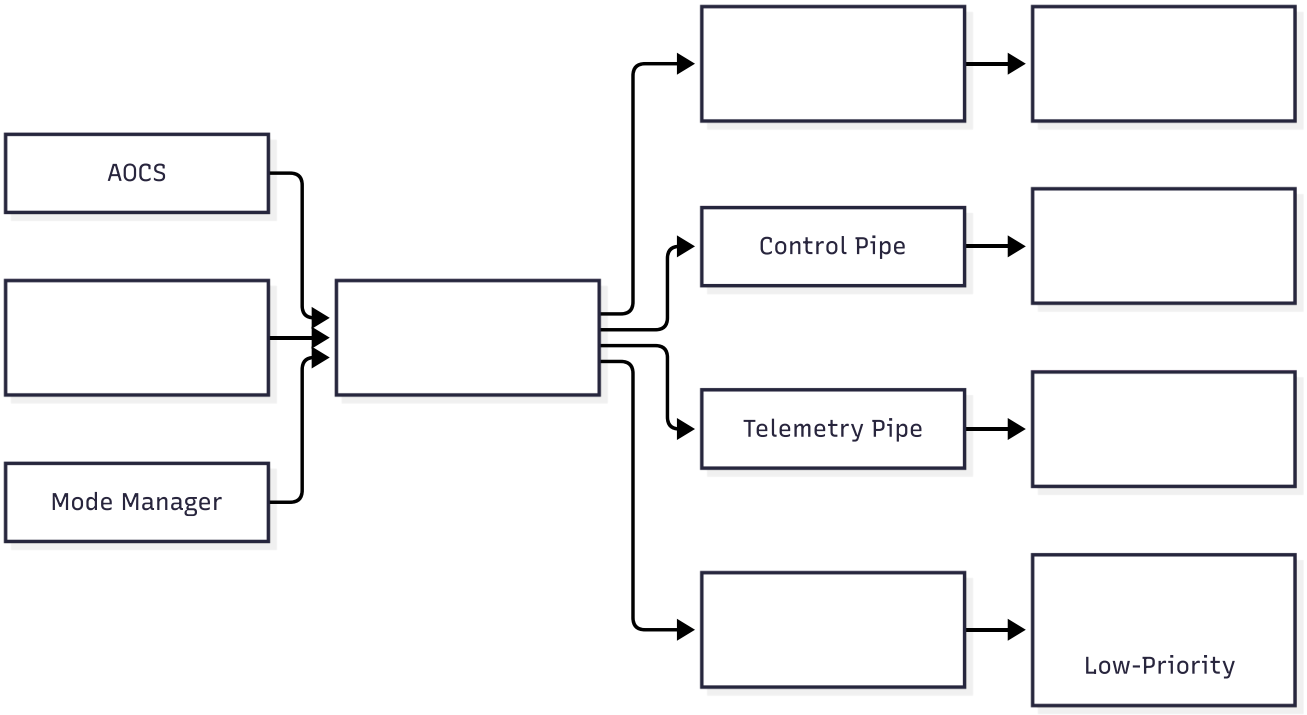
The Software Bus should not be described as automatically prioritising one message inside a pipe over another. Messages in a cFE pipe are normally received in send order.

Critical traffic is prioritised through the architecture by using:

- Separate Software Bus pipes
- Separate consuming tasks
- Higher FreeRTOS task priority for critical consumers
- Smaller and bounded critical messages
- Rate limits on debug and bulk telemetry
- Separate communication queues for UHF and S-band
- Explicit drop or overwrite policies for non-critical data

For example, a Health Monitor should not share a long-running payload-processing loop. It should have its own pipe and a higher-priority task. This ensures that health events are processed quickly without depending on message reordering inside a single FIFO pipe.

A subsystem interface should also avoid blocking for many seconds while holding application resources.



3.3 Event Services

Event Services provide a standard way for applications to report operational and diagnostic events.

Applications use event identifiers and event types rather than independent `printf()` implementations. Representative event classes are:

- Debug
- Information
- Warning
- Error
- Critical fault

During flatsat and AIT, events may be displayed on a UART or CLI console by a diagnostic application. In flight, the same events may be:

- Sent through Telemetry Output
- Stored by Data Storage
- Used by Health Monitoring
- Filtered by event identifier
- Rate-limited

A typical application call is based on the cFE EVS API, for example:

```
        CFE_EVS_SendEvent(
            PROPULSION_TIMEOUT_EID,
            CFE_EVS_EventType_ERROR,
            "Propulsion response timeout, state=%u",
            (unsigned int)state);
```

Event generation must be rate-limited so that a failing device cannot saturate the telemetry link or consume excessive CPU time.

Table Services may be used to configure which event IDs are enabled, filtered or forwarded. This allows detailed debug information to be enabled for a specific subsystem during a contact without rebuilding the flight image.

Early boot failures that occur before Event Services are available should be written to a small system log or persistent boot record. cFE Executive Services provides a system-log mechanism for events that occur before EVS can be trusted.

3.4 Time Services

Time Services provide consistent time information to the applications.

The architecture distinguishes between:

- **Monotonic execution time**, derived from a stable hardware timer and used for control-loop periods, timeouts.
- **Mission elapsed time**, used for spacecraft-relative operations (i.e in 5000ms)
- **Spacecraft absolute time**, correlated with UTC or TAI through ground, GNSS, an RTC or another external source (API uses time.h)
- **Time quality**, indicating whether the current absolute time is valid and how it was obtained. (some apps/cmds may need better quality)

Time Services are responsible for:

- Time correlation
- Spacecraft-time distribution
- Timestamp generation
- Time-quality reporting
- External time synchronisation
- PPS handling or distribution, where implemented in software
- Support for time-tagged operations

Control loops use monotonic time. Ground-correlated absolute time is used for operations, telemetry and scheduling.

A change in absolute spacecraft time must not disturb periodic task execution. For example, correcting UTC by several seconds must not cause the AOCS task to execute twice or skip a control cycle.

The hardware PLL may be part of the clock-generation or clock-discipline implementation, but it is not itself the software time source. The software time source is the selected hardware timer, counter, RTC or external timing input exposed through the PSP and Time Services.

3.5 Table Services

Table Services manage configuration data that may change during the mission without recompiling the software.

Examples include:

- AOCS gains
- Sensor calibration data
- Health thresholds
- Rate-of-change thresholds
- Fault persistence values
- Communication timeouts
- Subsystem addresses
- Telemetry selection and rate configuration
- Mode-transition parameters
- Propulsion constraints
- Scheduler definitions
- Event filters

Tables contain configuration, not operational telemetry. Applications may generate telemetry reporting the active table version, validation status or selected parameter values, but telemetry messages are not themselves stored in cFE tables.

Each table should have:

- Version information
- Size
- Integrity check
- Compatibility identifier
- Validation function

- Default values
- Fallback behaviour

The application normally reads the active table during operation. Ground or an onboard application can load an inactive candidate table, validate it and request activation. The application controls when the update becomes active so that configuration cannot change at an unsafe point in the control cycle.

This is particularly useful for AOCS gains, FDIR thresholds and communication configuration.

3.6 Mode Manager

The Mode Manager is the authoritative owner of the global spacecraft mode.

Representative modes are:

- Boot
- Recovery
- Safe
- Standby
- Nominal
- Payload Operations
- Propulsion Preparation
- Propulsion Execution
- Software Update

Only the Mode Manager authorises spacecraft-level transitions.

Applications may:

- Request a transition
- Report a fault
- Report that a transition condition is satisfied
- Reject a requested operation because of local state

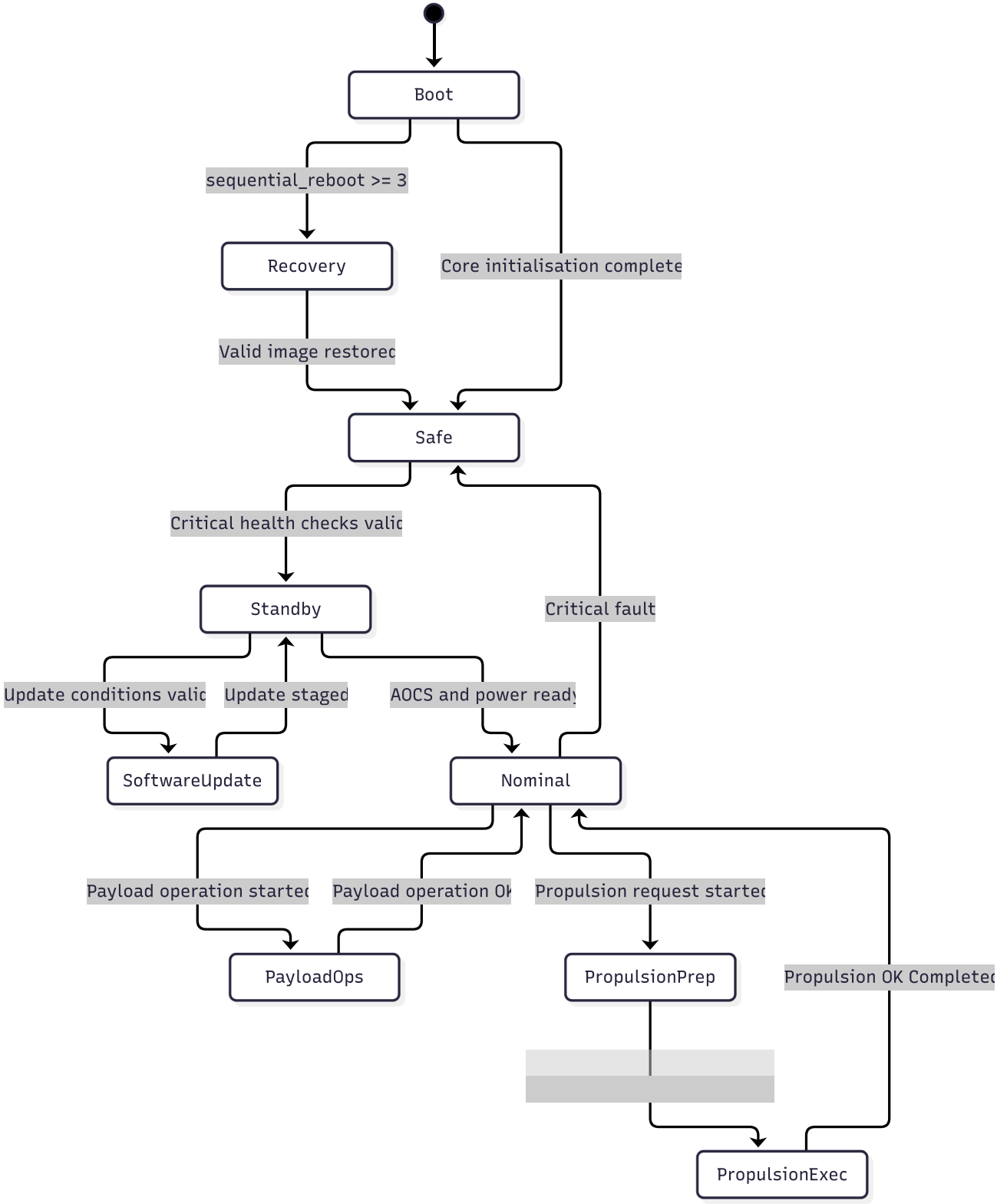
Applications cant change the global spacecraft mode.

The Mode Manager should remain a small and explicit state machine. It evaluates:

- Current mode
- Requested mode
- Health status
- Active faults

- Power and thermal conditions
- AOCS status
- Propulsion inhibits
- Communication requirements
- Transition-specific guards

Applications subscribe to the current-mode message and keep a local read-only copy. They do not need to implement every global mode internally. Each application defines which commands or behaviours are allowed in each relevant spacecraft mode.



This diagram is intentionally simplified and without the error pathz. The final transition table should also have specific conditions.

This mode manager is easy to trace to requirements and statuses, and can be checked by systems and QA without any coding experience.

3.7 Command Ingest

Command Ingest receives telecommands from the communication layer and converts them into normalised Software Bus commands.

Its responsibilities include:

- Receiving a decoded frame or packet from the link layer
- Validating packet length and structure
- Checking command service and subtype
- Translating the packet into an internal command
- Publishing the command on the Software Bus
- Generating command acceptance reports

CRC, RDP and link-level retransmission are normally responsibilities of the underlying link or transport layer. Command Ingest may verify an end-to-end packet CRC if the selected packet format includes one, but it should not duplicate every lower-layer protocol mechanism.

Parameter checking is divided into two levels:

- Command Ingest checks common structural limits, lengths and encoding.
- The destination application checks mission semantics and current-state constraints.

For example, Command Ingest can verify that a propulsion duration is correctly encoded and within the representable field range. The Propulsion Application and Mode Manager decide whether that duration is allowed in the current mode, thermal state and propulsion state.

For example, something convoluted that can be done : the ground interface is PUS-compatible. CSP may be used as an onboard transport.

```
Ground command
→ UHF or S-band link
→ Link and transport validation
→ PUS-compatible decoding
→ Command Ingest
→ Software Bus command
→ Destination CSP application
```

File transfer can be handled by a dedicated transfer application rather than making the normal command parser process large binary transfers. This keeps ordinary commanding small and predictable. CFDP in cFS provides something like that.

3.8 Telemetry Output

Telemetry Output subscribes to configured Software Bus messages and prepares them for ground transmission.

Its responsibilities include:

- Subscribing to selected internal telemetry
- Adding timestamps and sequence counters
- PUS-compatible packetisation
- Selecting or requesting the appropriate ground link
- Applying rate and priority configuration
- Sending command-verification reports
- Separating critical telemetry from bulk data

Representative telemetry classes are:

CLASS	EXAMPLES	PREFERRED HANDLING
Critical	Safe-mode state, propulsion fault, boot failure	UHF, highest priority
High	Command verification, AOCS state, power status	UHF or configured nominal link
Normal	Housekeeping and subsystem telemetry	Scheduled nominal downlink
Bulk	Payload files, detailed logs, memory dumps	S-band or other high-rate link

UHF is assumed to remain available for nominal commanding, housekeeping and contingency recovery. S-band is preferred for payload data, detailed logs and large software-image transfers.

Telemetry Output should not block critical applications while waiting for a radio. Applications publish messages and continue; the communications services manage bounded queues, link availability and drop policies.

3.9 Mission Scheduler

The Mission Scheduler manages planned and time-tagged operations.

It supports:

- Absolute time-tagged commands
- Relative sequences

- Periodic operations
- Deferred commands
- “maybe” conditional requests

The scheduler does not execute the complete operation inside its own task. It publishes a Software Bus command to the destination application, and that application performs the operation asynchronously.

```
Scheduled time reached
→ Scheduler publishes command
→ Destination application validates current state, in case it cant start.
→ Application starts operation, tells scheduler when finished and return code.
```

Brief mission sequencing may be represented in the scheduler, but FDIR recovery policy and spacecraft-level transitions remain under the Health Monitor, FDIR and Mode Manager.

NASA’s Stored Command application is an existing example of absolute and relative time-tagged command sequences.

uPython is also used in some ESA missions, but I don’t know how big of a problem is to implement.

3.10 AOCS Application

The AOCS Application runs the state-estimation and control algorithms.

It does not directly poll every sensor bus inside the model step. Instead:

1. Subsystem Interface Applications acquire and decode sensor data (Sensor Driver)
2. They publish normalised, timestamped measurements through the Software Bus.
3. The AOCS Application subscribes to those messages and maintains the latest coherent input set.
4. At each periodic release, it validates data age and validity.
5. It executes the model step.
6. It publishes normalised actuator requests through the Software Bus.

The AOCS model runner may call functions generated by Embedded Coder, for example:

```
        rtw_Model_initialize();
        rtw_Model_step();
```

A wrapper around the autogenerated code is responsible for:

- Mapping Software Bus data into model inputs
- Validity flags

- Timestamp and data-age checks
- Model initialisation
- Execution-time profiling
- Publishing actuator commands
- Reporting faults and deadline misses

A representative input type is:

```
        typedef struct {
            uint64_t timestamp_us;
            bool valid;
            float value_imu[3];
            float value_etc;
        } aocs_vector_measurement_t;
```

Protection logic that is outside the AOCS mathematical model should remain in manually developed software. This includes stale-data handling, hard inhibits, deadline supervision and fallback-state selection.

> [!assumption] Note
>
> a lot of extra development can be done on AOCS side. For HIL testing, implementing another application that lets matlab work directly with the Bus Sensor may be done, while on HIL mode The cFS adjancent software, NOS3, is a lab that uses the primitives defined in the system to it can be interfaced to a PC running simulations. I dont know much about that.

3.11 Bootloader and OTA Manager

The complete boot, recovery and rollback strategy is defined in Section 1.4. This section only defines the component.

The **bootloader**:

- Runs before FreeRTOS and cFE
- Selects and validates image
- Starts it
- Performs rollback selection after failed boots.

The **Golden Recovery Image**:

- Provides minimal ground communication
- Provides essential storage and repair commands
- Receives replacement images

- Safes the spacecraft as required

The **Update Manager application** runs inside the operational or recovery image and is responsible for:

- Starting an update session
- Selecting the inactive slot or operational image area
- Receiving file blocks
- Validating block sequence and offset
- Supporting transfer resume
- Checking complete-image integrity (xxhash)
- Recording build and version metadata
- Reporting update status
- wait for controlled activation

The Update Manager cannot overwrite the currently executing validated image in the A/B configuration.

```
 const uint32_t image_address =
    (boot_selection == BOOT_IMAGE_APP)
        ? APP_IMAGE_ADDRESS`
        : GOLDEN_IMAGE_ADDRESS;

if (image_is_valid(image_address, image_size)) {
    run_image(image_address);
}
```

A commit ID may be included in the image metadata for traceability. A cryptographic hash is used for integrity.

3.12 Driver-Independent Subsystem Library and Software Bus Integration

The driver-independent subsystem architecture includes the Software Bus explicitly.

Mission applications communicate through normalised Software Bus messages. Hardware Interface Applications translate those messages into subsystem-library calls and publish normalised telemetry back onto the Software Bus.

The mission-facing subsystem API :

```
        typedef enum {
            SUBSYSTEM_OK = 0,
            SUBSYSTEM_ERR_ARGUMENT,
            SUBSYSTEM_ERR_TIMEOUT,
        } subsystem_status_t;

        typedef struct {
            uint32_t timeout_ms;
            uint8_t retries;
            uint16_t node_id;
            void *transport_context;
        } subsystem_context_t;

        subsystem_status_t command_send(
            subsystem_context_t *context,
            const void *command,
            size_t command_size);

        subsystem_status_t telemetry_read(
            subsystem_context_t *context,
            void *telemetry,
            size_t telemetry_size);

        subsystem_status_t subsystem_get_status(
            subsystem_context_t *context,
            void *status);

        subsystem_status_t subsystem_reset_link(
            subsystem_context_t *context);
```

A generic transport interface may be:

```
typedef struct {
    int (*open)(void *context);
    int (*close)(void *context);

    int (*write)(
        void *context,
        const uint8_t *data,
        size_t length,
        uint32_t timeout_ms);

    int (*read)(
        void *context,
        uint8_t *data,
        size_t maximum_length,
        uint32_t timeout_ms);

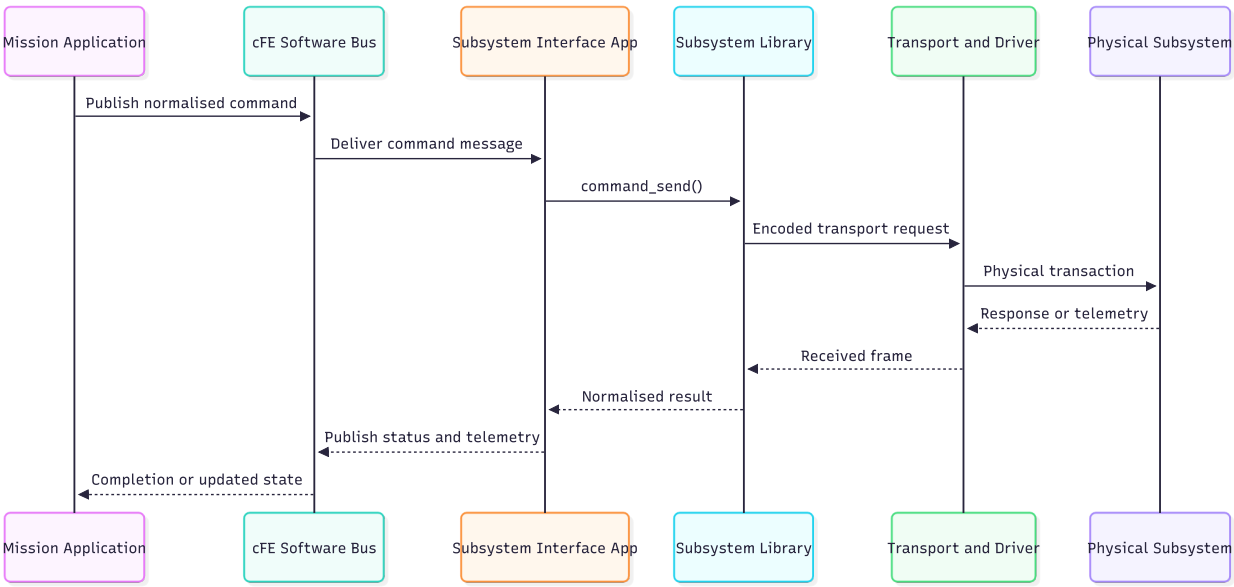
    int (*reset)(void *context);
    int (*get_status)(void *context, void *status);
} transport_interface_t;
```

Possible platform backends are:

ENVIRONMENT	BACKEND EXAMPLE
FreeRTOS	BSP UART, SPI or CAN driver with queues and DMA
CMSIS	`Driver_USART`, `Driver_CAN`, `Driver_SPI`
Zephyr	UART, CAN, SPI device API, RTOS `but linux-like`
Linux	`/dev/ttyS0`, SocketCAN or sockets
Host simulation	Loopback, cmock or virtual device/ function
Distributed spacecraft	CSP transport adapter

The Subsystem Interface Application owns the Software Bus pipes and the asynchronous transaction state. It should not make a high-priority mission application block directly on a slow UART or remote transaction.

A representative command interaction is:



The cFE Software Bus allows other cFS applications, such as Health Monitoring, Data Storage and Telemetry Output, to observe the same normalised subsystem telemetry without coupling themselves to the device driver.

---

4. Scheduling and Real-Time Behaviour

The software runs on a single-core processor using the FreeRTOS fixed-priority, pre-emptive scheduler.

Task priorities are assigned according to timing criticality. A high-level functional importance does not automatically mean a high RTOS priority; priority is based on the required response time, deadline and possible consequence of delayed execution.

The proposed priority groups are:

PRIORITY GROUP	MAIN FUNCTIONS
Critical	Emergency inhibits, watchdog coordination and time-critical hardware actions
Very high	Sensor acquisition, AOCS model execution and actuator output
High	Propulsion sequencing and propulsion-state monitoring
Medium-high	Health monitoring, Command Ingest and Mode Manager
Medium	Mission Scheduler, housekeeping generation and subsystem coordination
Low	File management, payload buffering and downlink preparation
Background	Software upload, log processing, checksums and diagnostics

The exact task periods depend on the final AOCS and propulsion requirements. Initial values can be assigned during prototyping and replaced with measured values after target-hardware profiling.

TASK	INITIAL SCHEDULING APPROACH
Sensor acquisition	Fixed-rate periodic task
AOCS model runner	Fixed-rate periodic task
Actuator output	Fixed-rate periodic or synchronised with AOCS
Propulsion sequencer	Event-driven state machine with timed transitions
Health monitoring	Multi-rate periodic monitoring
Command Ingest	Event-driven with bounded processing
Mission Scheduler	Periodic release of stored or time-tagged commands
Housekeeping	Low-rate periodic task
Software update	Background task, suspended during critical operations

Periodic tasks should use absolute releases, for example `vTaskDelayUntil()`, rather than delaying relative to the end of the previous execution. This avoids accumulated timing drift.

```
        static void aocs_task(void *argument)
        {
            TickType_t next_release = xTaskGetTickCount();

            for (;;)
            {
                aocs_execute_cycle();

                vTaskDelayUntil(
                    &next_release,
                    pdMS_TO_TICKS(AOCS_PERIOD_MS));
            }
        }
```

Interrupt Service Routines should only perform the minimum required work:

- Capture a timestamp
- Read or clear hardware status
- Complete a DMA transaction
- Store received data in a bounded buffer
- Notify the corresponding task

Packet parsing, retries, model execution, file access and detailed event generation should be performed in task context.

The critical control path is:



Critical tasks shall have:

- A defined period and deadline
- A bounded input queue
- A measured execution time
- A monitored stack allocation
- A defined response to a missed deadline
- No unbounded file or communication operations

The cFE Software Bus pipes are sized according to the expected message rate and worst-case consumer latency. Critical traffic should use dedicated pipes and appropriately prioritized consumer tasks rather than assuming that one message can overtake another inside the same FIFO pipe.

The target is to keep nominal CPU utilization below approximately 45%, with planned worst-case utilization below 60%. These are initial design targets rather than fixed requirements and must be confirmed through measurement on the final OBC.

---

5. Communication Architecture

The communication architecture separates four concerns:

1. Ground TM/TC services
2. Internal OBC communication
3. Communication with distributed spacecraft nodes
4. Physical buses and radio links

The selected baseline is:

```
Ground monitoring and control
→ PUS-compatible TM/TC

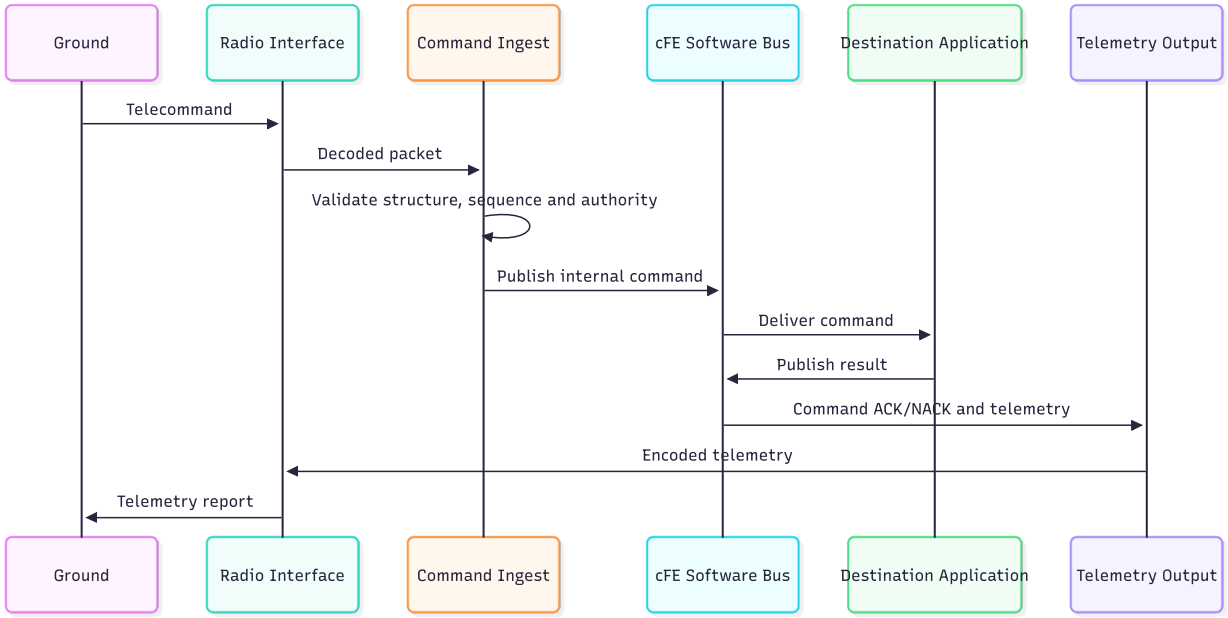
Communication between applications inside the OBC
→ cFE Software Bus

Communication with distributed smart nodes
→ CSP or a subsystem-specific protocol

Physical transport
→ CAN, UART, RS-422, SpaceWire or another selected interface
```

PUS is used as a mission-tailored service model. Only the services required by the mission need to be implemented. Typical functions include command verification, housekeeping, event reporting, scheduling, parameter management and file transfer.

The ground command path is:



Command Ingest validates common packet properties. The receiving application validates whether the requested action is allowed in its current state.

For example, the Propulsion Application and Mode Manager still verify:

- Inhibit status
- Active faults
- Power and thermal conditions
- Valid execution window (if command is outdated)

Then, there is some error at some points the Propulsion App may output the error status to the telemetry.

5.1 Subsystem communication



This architecture allows an implementation to use FreeRTOS drivers on the flight OBC and Linux device files or mocked transports during development without changing the mission application.

CSP/PUS is appropriate when a remote subsystem is an intelligent embedded node. It remains below the mission-facing subsystem API.

5.2 UHF and S-band

The architecture assumes two logical communication classes.

UHF or equivalent low-rate link

- Nominal telecommands
- Basic housekeeping
- Command verification
- Safe-mode telemetry
- Contingency access
- Recovery commands

S-band or equivalent high-rate link

- Payload files
- Detailed logs
- Memory dumps
- Software images
- Large table uploads
- Bulk engineering telemetry

The spacecraft must not depend on the high-rate link for basic recovery. A minimum command and telemetry capability should remain available through the low-rate link.

Bulk transfers are block-based, resumable and rate-limited. They execute at lower priority than AOCS, propulsion and critical telemetry.

Selected cFS applications can support this architecture, or custom ports of uFS, FTP, BTP, etc.

- HK for configurable housekeeping packets
- DS for storing selected Software Bus messages
- CFDP for file transfer and controll

---

6. Fault Containment and Recovery

The selected FDIR architecture is hybrid, thats why it couldnt be written as a specific app, since it happens in every task, and the results of these tasks are sent upon a central task.

Applications and subsystem interfaces detect local failures because they understand their own data, state and timing requirements. Central Health Monitoring, FDIR and the Mode Manager coordinate recovery actions that affect the complete spacecraft. If, some of this local faults is minimal, (i.e some low priority hk corrupted), the HK app can handle it, and retry the operation without giving notice to the main FDIR app.

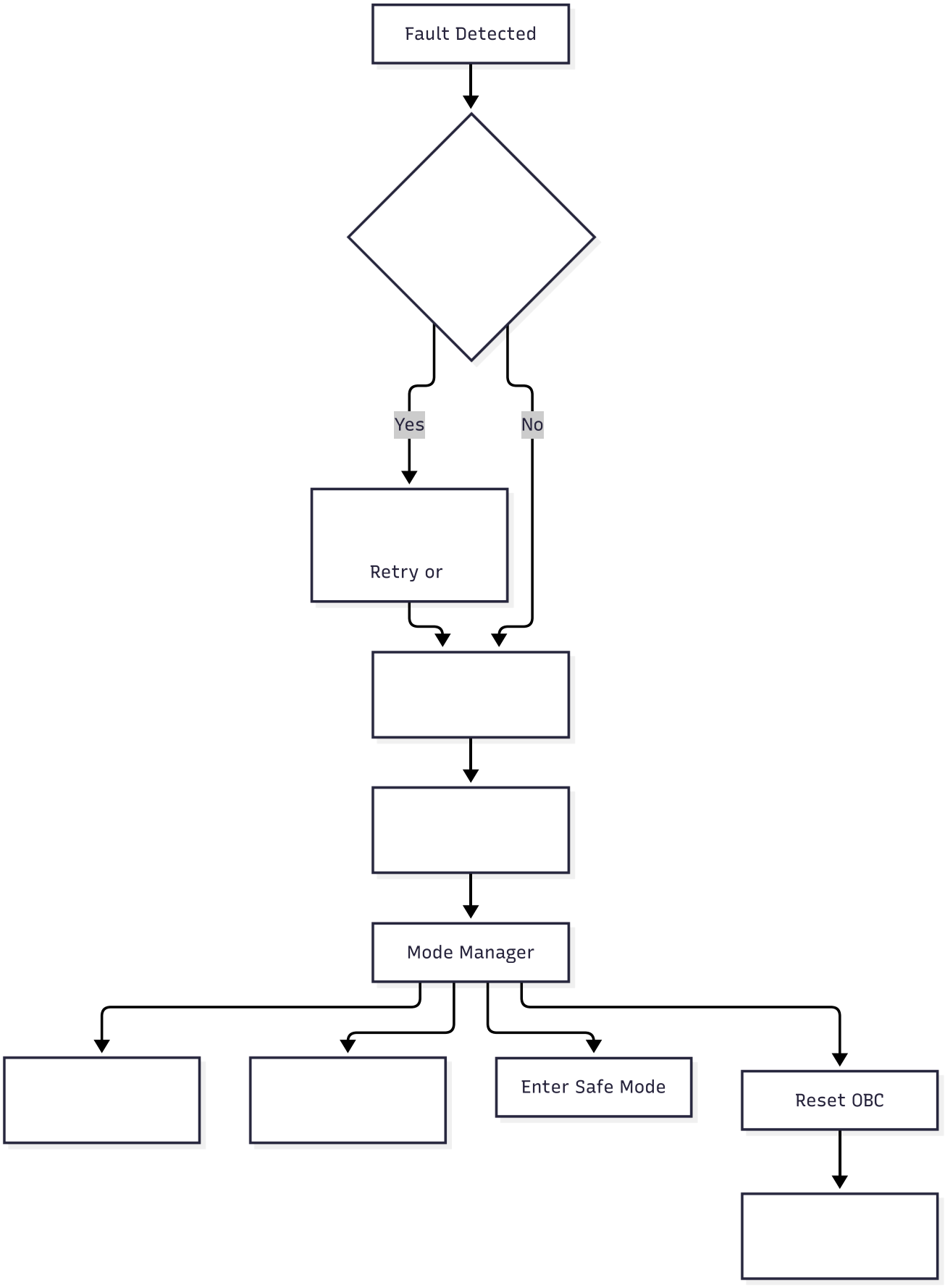
Local detection examples include:

- Missing or invalid sensor samples
- Communication timeout
- Protocol or CRC error
- Stale AOCS input
- AOCS deadline overrun
- Invalid propulsion-state transition
- Storage failure
- Queue overflow
- Application heartbeat failure
- Invalid update image

If needed, a central FDIR can be developed that evaluates.

- Severity
- Persistence
- Number of repetitions
- Current spacecraft mode
- Related active faults
- Previous recovery attempts
- Available recovery options

Local applications may perform bounded and reversible actions such as retrying a transaction or resetting a communication interface. Spacecraft-level actions are coordinated by the Mode Manager.



The cFS Health and Safety application can provide execution monitoring and watchdog servicing. Limit Checker can evaluate configured thresholds. These applications provide detection mechanisms; the mission-

specific FDIR and Mode Manager still own the recovery policy.

Critical fault evidence should be stored in a protected circular log containing :

- Reset reason
- Active software version
- Spacecraft mode
- Last critical events
- Task deadline status (and other FreeRTOS measurements)
- AOCS state
- Watchdog state
- Stack info (PC, REGS, WD,) a core-dump
